

Université de Sherbrooke

Département d'informatique

IFT814 - Cryptographie

Devoir 1

Groupe 10

Automne 2026

Enseignant : Martin Fiset

Fiche de paramètres utilisée pour les calculs : Equipe_03.txt.

Membres de l'équipe

Rodier Marcel Tumamo Simo

Blakoua Adja

Cheikh Ahmadou Tidiane Ly

Odounlami Lucien Gbessemehlan

Date : 2 octobre 2026

Déclaration d'usage des outils d'IA générative

Outil utilisé : ChatGPT / Codex, le 2 octobre 2026. L'assistance a porté sur l'analyse de l'énoncé et des supports de cours, l'explication des notions, la génération de code Python et de texte, les calculs, les vérifications par exécution et la mise en forme du rapport. Le présent document contient du code, des résultats et des passages rédigés avec cette assistance. Les vérifications informatiques mentionnées correspondent aux contrôles exécutés lors de sa préparation.

Conventions et reproductibilité

L'alphabet suit $a = 0, \dots, z = 25$. Les indices commencent à 0. Les calculs utilisent les entiers Python, une source aléatoire sûre (secrets), et des fonctions standard. Aucun chiffrement ni MAC préimplémenté n'est appelé.

```
m = ceciestlemessageclairadechiffrer
```

Le message compte 32 lettres. Pour le binaire, chaque lettre occupe exactement 5 bits, la première lettre dans les bits de poids fort. Les zéros initiaux sont conservés dans les affichages. Un bloc de valeur 26 à 31 est refusé lors du décodage en lettres.

L'annexe contient le code source complet. Le fichier de paramètres doit être placé à côté du script. Les générateurs sont réellement appelés : les exemples aléatoires changent d'une exécution à l'autre, contrairement aux traces imposées.

```
python solution_ift814.py > resultats.json
```

Donnée	Empreinte SHA-256, 8 caractères
Substitution, minuscules	2b86f4d7
c1 c2, chaînes hexadécimales	d7b627e1
Vigenère, minuscules	768d2dfa

Pour ces fichiers, les empreintes portent sur les chaînes normalisées ci-dessus, et non sur les octets obtenus avec `bytes.fromhex`. `hashlib` ne sert qu'au contrôle de copie.

1.1 César à double décalage

1. Code et exemples

Gen1 tire indépendamment les deux décalages uniformes de 0 à 25. E1 ajoute k_1 aux indices pairs et k_2 aux indices impairs. D1 soustrait les mêmes valeurs. La position est essentielle : les deux décalages ne s'additionnent pas sur chaque lettre.

```
E1(k,m)[i] = (m[i] + k[i mod 2]) mod 26
D1(k,c)[i] = (c[i] - k[i mod 2]) mod 26
```

Code : fonctions Gen1, E1 et D1, en annexe. Exemple effectif avec une clé générée :

```
{
  "k": [
    13,
    1
  ],
  "c": "pfpjrtgmrnrftbtfpmnjebqfpivgssrs",
  "m_retrouve": "ceciestlemessageclairadechiffrer"
}
```

2. Trois traces imposées

Exécution 1

```
Alice : m = ceciestlemessageclairadechiffrer
        k = [2, 20]
        c = eyecgmvgggmuuiyefcctufyebkzhlg
Ève   : c = eyecgmvgggmuuiyefcctufyebkzhlg
Bob   : c = eyecgmvgggmuuiyefcctufyebkzhlg
        k = [2, 20]
        m = ceciestlemessageclairadechiffrer
```

Exécution 2

```
Alice : m = ceci est le message clair à déchiffrer
        k = [19, 0]
        c = vevixsmlxmxslazevltikawevhbfyrxr
Ève   : c = vevixsmlxmxslazevltikawevhbfyrxr
Bob   : c = vevixsmlxmxslazevltikawevhbfyrxr
        k = [19, 0]
        m = ceci est le message clair à déchiffrer
```

Exécution 3

```
Alice : m = ceci est le message clair à déchiffrer
        k = [20, 8]
        c = wmwqyantyuyamiamwtuqlixmwpcnzzyz
Ève   : c = wmwqyantyuyamiamwtuqlixmwpcnzzyz
Bob   : c = wmwqyantyuyamiamwtuqlixmwpcnzzyz
        k = [20, 8]
        m = ceci est le message clair à déchiffrer
```

3. Attaque d'Ève

Eve1 teste toutes les 26×26 clés et déchiffre chaque candidat. Un petit lexique français, déclaré dans le code avant l'attaque et ne contenant pas le message entier, sert à rechercher une segmentation complète du texte. La programmation dynamique vérifie si tous les caractères peuvent être couverts par des mots du lexique. Ce critère est reproductible mais volontairement limité ; un grand dictionnaire ou un score linguistique donnerait potentiellement d'autres candidats.

Exécution	Clé retrouvée	Candidats retenus par le critère
1	[2, 20]	1
2	[19, 0]	1
3	[20, 8]	1

Segmentation retenue : ceci est le message clair à déchiffrer

Le programme teste effectivement 676 clés par cryptogramme, soit 2 028 tests pour les trois traces. Il n'utilise pas une comparaison avec le message de référence pour sélectionner le candidat. Le nombre de candidats plausibles est ici 1 selon ce lexique ; ce n'est pas une preuve d'unicité absolue parmi toutes les phrases françaises.

4. Comparaison de sécurité

Le César classique a 26 clés, soit environ 4,70 bits. Le schéma alterné en possède 676, soit environ 9,40 bits. Une recherche exhaustive coûte 26 fois plus de déchiffrements, mais demeure immédiate sur un texte aussi court. Le gain réel de sécurité reste faible.

Avec trois décalages appliqués cycliquement aux trois classes d'indices modulo 3, l'espace compte $26^3 = 17\,576$ clés, soit environ 14,10 bits. Il reste très petit. Si la formulation signifie plutôt ajouter k_3 aux positions 0, 3, 6, ... tout en conservant l'alternance k_1/k_2 , le motif résultant est de période 6 et comporte toujours seulement 26^3 choix de clés. Cette variante ne change donc pas la conclusion.

1.2 Substitution monoalphabétique

1. Code et exemples

Gen3 applique Fisher-Yates en parcourant la permutation de droite à gauche et en échangeant la position i avec une position uniforme dans $0..i$. La source `secrets.randbelow` évite le biais d'un simple modulo sur une valeur aléatoire. Les $26!$ permutations ont ainsi la même probabilité. E3 utilise l'image de chaque lettre. D3 construit explicitement la permutation inverse.

```
{
  "k": "xovetyrdbpwasinumzclfqjgkh",
  "c": "vtvbtclatstccxrtvaxbzxetvdbyyztz",
  "m_retrouve": "ceciestlemessageclairadechiffrer"
}
```

2. Traces imposées

Exécution 1

```
Alice : m = ceciestlemessageclairadechiffrer
        k = rsaofhvpmtqzbkcigynxldeju
        c = afamfynqfzfyyrvfaqrmgrofapmhhgfg
Ève   : c = afamfynqfzfyyrvfaqrmgrofapmhhgfg
Bob   : c = afamfynqfzfyyrvfaqrmgrofapmhhgfg
        k = rsaofhvpmtqzbkcigynxldeju
        m = ceciestlemessageclairadechiffrer
```

Exécution 2

```
Alice : m = ceciestlemessageclairadechiffrer
        k = ixodqfulgkaenhscjbtvmpzyrw
        c = oqogqtveqnqttiuqoeigbidqolgffbqb
Ève   : c = oqogqtveqnqttiuqoeigbidqolgffbqb
Bob   : c = oqogqtveqnqttiuqoeigbidqolgffbqb
        k = ixodqfulgkaenhscjbtvmpzyrw
        m = ceciestlemessageclairadechiffrer
```

Exécution 3

```
Alice : m = ceciestlemessageclairadechiffrer
        k = jwxqkcgyhftoudpvrisenmabzl
        c = xkxhkseokukssjgkxojhijqkxyhcciki
Ève   : c = xkxhkseokukssjgkxojhijqkxyhcciki
Bob   : c = xkxhkseokukssjgkxojhijqkxyhcciki
        k = jwxqkcgyhftoudpvrisenmabzl
        m = ceciestlemessageclairadechiffrer
```

3. Fréquences et démarche itérative

Lettre chiffrée	Nombre	Observée (%)	Lecture	Référence (%)
A	68	17.09	e	14.7
G	40	10.05	s	7.9
E	32	8.04	u	6.3
F	31	7.79	r	6.7
K	29	7.29	n	7.1
P	27	6.78	t	7.2
W	23	5.78	l	5.5
D	22	5.53	o	5.8
J	21	5.28	i	7.5
I	21	5.28	a	7.6
Y	16	4.02	d	3.7
B	15	3.77	c	3.3
R	14	3.52	m	3
O	11	2.76	p	2.5
T	7	1.76	h	0.7
Z	4	1.01	q	1.4
S	4	1.01	v	1.8
C	4	1.01	f	1.1
H	3	0.75	g	0.9
X	2	0.50	x	0.4
U	2	0.50	j	0.6
Q	1	0.25	y	0.1
V	1	0.25	b	0.9
L	0	0.00	absente	-
M	0	0.00	absente	-
N	0	0.00	absente	-

La lettre A atteint environ 17 %, ce qui suggère e, contre 14,7 % dans la table de référence. Les écarts à la référence ne constituent pas des erreurs : le texte a seulement 398 lettres. Le classement des fréquences sert à proposer des hypothèses, pas à imposer une correspondance mécanique.

Bigramme chiffré	Nombre	Lecture finale
AG	14	es
WA	13	le
YA	10	de

Bigramme chiffré	Nombre	Lecture finale
AF	9	er
KP	9	nt
EF	8	ur
AB	7	ec
AK	7	en
IK	7	an
JG	6	is
GG	6	ss
KA	6	ne
AP	6	et
BT	6	ch
RA	6	me

Étape 1 : une proposition fondée sur les répétitions

On propose $A \rightarrow e$ et $G \rightarrow s$. La séquence WAG, fréquente, pourrait alors correspondre à les, d'où $W \rightarrow l$. À ce stade, les autres caractères restent inconnus. Ces choix sont des hypothèses qui doivent être confrontées aux mots et aux groupes de lettres.

_e_s_ess_e_lesles_e_e_ _e_les_ _l_ _es_e_ _e_ _s_e_ _ele_s_ _ess_s_ _ele_e_ _
 _sselesl_e_ _le_e_ _e_e_les_e_ _e_ _e_s_ _es_e_ _e_ _se_e_ _e_ _
 l_ _e_ _le_ _e_ _e_ _es_s_ _e_ _ss_ _sse_l_s_e_ _ss_s_ _
 _e_es_e_ _e_e_ _l_ses_ _s_ _e_e_ _lesele_es_ _e_s_ _s_ _e_ _el_ _ll_ _
 e_ _se_e_ _le_ _e_s_ _le_ls_ _el_e_ _

Étape 2 : reconnaissance de « depuis des siècles »

Le début devient compatible avec « depuis des siècles les... ». On ajoute $Y \rightarrow d$, $O \rightarrow p$, $E \rightarrow u$, $J \rightarrow i$ et $B \rightarrow c$. La répétition G G donne ss dans « des siècles ». La lecture cohérente du début apporte un argument plus fort que le seul classement des fréquences.

depuisdesieclesles_e_e_u_e_lesdipl_ _esc_e_c_e_ _s_e_ _eleu_s_d_ess_s_uele_e_i
 puisselesli_eu_p_ _c_ _ledec_ _edeclespe_e_deu_pe_s_ _esdec_ _e_i_du_sec_e_c_u_e_p_ _
 l_ _de_ _u_ _le_ _deu_ _di_ _eu_u_ _i_uesu_ _is_ _e_ _puiss_ _p_u_ _i_c_ _sse_ _plusieu_ss_s_ _
 _e_esdec_i_ _e_e_ _u_ _ilises_u_ _u_d_uisu_i_ _e_e_ _lesele_es_ _c_e_us_u_us_ _e_del_c_lli_ _
 ep_u_ _se_e_ _lec_uc_e_dus_leilsu_ _u_el_e_i_ _

Étape 3 : extension et correction par le contexte

Le segment HAKAFIEXAP se lit « generauxet ». Il impose $H \rightarrow g$, $K \rightarrow n$, $F \rightarrow r$, $I \rightarrow a$, $X \rightarrow x$ et $P \rightarrow t$. $I \rightarrow o$ aurait donné « generouxet » ; la correction $I \rightarrow a$ rétablit « generauxet ». « diplomates », « transmettre », « protocole », « ordinateur quantique », « suffisamment » et « aujourd'hui » permettent ensuite de compléter les correspondances observées. Ces trois états décrivent des hypothèses successives et leur validation linguistique, sans prétendre à une attaque purement automatique.

depuisdesiecleslesgenerauxetlesdiplomatescherchentatransmettreleursordressansquelennemi
 puisseleslireunprotocoledechangedeclespermetadeuxpersonnesdeconvenirdunsecretcommunenpar
 lantdevanttoutlemondeunordinateurquantiquesuffisammentpuissantpourraitcasserplusieurssys
 temesdechiffrementutilisesaujourdhuisurinternetleselevesontmarchejusquausommetdelacollin
 epourobserverlecoucherdusoleilsurtoutelaregion

4. Texte clair

Depuis des siècles, les généraux et les diplomates cherchent à transmettre leurs ordres sans que l'ennemi puisse les lire. Un protocole d'échange de clés permet à deux personnes de convenir d'un secret commun en parlant devant tout le monde. Un ordinateur quantique suffisamment puissant pourrait casser plusieurs systèmes de chiffrement utilisés aujourd'hui sur Internet. Les élèves ont marché jusqu'au sommet de la colline pour observer le coucher du soleil sur toute la région.

La ponctuation et les espaces ci-dessus sont une restitution de lecture. La vérification utilise uniquement la chaîne sans accents ni espaces affichée à l'étape 3.

Clé complète compatible, et limite d'identification

Alphabet clair : abcdefghijklmnopqrstuvwxyz
Images : ivbyachtjulwrkdozfgpesmxqn

Les lettres chiffrées L, M et N sont absentes. Les lettres claires k, w et z le sont également. Le choix $k \rightarrow L$, $w \rightarrow M$, $z \rightarrow N$ complète la permutation, mais six complétions (3!) produisent le même cryptogramme. Cette clé est donc une clé complète compatible, sans garantie d'être la permutation d'origine sur les lettres absentes. Le rechiffrement exact a été vérifié.

5. Discussion

L'espace des clés contient $26! = 403,291,461,126,605,635,584,000,000$ permutations, soit environ $4,03 \times 10^{26}$ et 88.38 bits. L'analyse de fréquences contourne l'exploration exhaustive parce que la substitution conserve les répétitions, les motifs de mots et les statistiques des lettres.

Un texte de 32 lettres donne moins de répétitions et des fréquences moins fiables qu'un texte de 398 lettres. Sans connaissance du message ou information supplémentaire, Ève ne peut pas garantir le déchiffrement des courtes traces par la seule analyse statistique. Dans le contexte du devoir, si le message de référence est public et connu d'Ève, il constitue au contraire une information de texte clair connu ; il ne faut pas confondre cette situation avec une attaque à cryptogramme seul.

2.1 Masque jetable et secret parfait

1. Code et conversion

vers_bits concatène les blocs de 5 bits. Gen2 tire uniformément 160 bits. E2 calcule le XOR entre la clé et l'encodage du message ; D2 refait le même XOR, puis décode les blocs. Le paramètre de longueur conserve les zéros initiaux et permet aussi des messages de 48 lettres en partie 2.2.

```
{  
  "k": "cd3b44ab34e2f614d4312b415cdb2a6a1f5f0c42",  
  "c": "dc3fc6e15fc1ff3dd4f53981d45b4e7bcf0dc8d3",  
  "m_retrouve": "ceciestlemessageclairadechiffrer"  
}
```

2. Encodage du message commun aux trois traces

```
m = ceciestlemessageclairadechiffrer  
m_hex = 1104824a6b23092900c412c088806411d052c491  
m_bits = 00010001000001001000001001001010011010110010001100001001001010010000000  
01100010000010010110000001000100010000000011001000001000111010000010100101100010  
010010001
```

Exécution 1

```
Alice : m = ceciestlemessageclairadechiffrer
        m_hex = 1104824a6b23092900c412c088806411d052c491
        k = 48f42f09be44cd985c7b8094f0be6f0c53b20f06
        c = 59f0ad43d567c4b15cbf9254783e0b1d83e0cb97
Ève : c = 59f0ad43d567c4b15cbf9254783e0b1d83e0cb97
Bob : c = 59f0ad43d567c4b15cbf9254783e0b1d83e0cb97
        k = 48f42f09be44cd985c7b8094f0be6f0c53b20f06
        m = ceciestlemessageclairadechiffrer
```

Exécution 2

```
Alice : m = ceciestlemessageclairadechiffrer
        m_hex = 1104824a6b23092900c412c088806411d052c491
        k = 13f6b41cb7c47c2ea63f8fe5fd14be142b6b8df7
        c = 02f23656dce77507a6fb9d257594da05fb394966
Ève : c = 02f23656dce77507a6fb9d257594da05fb394966
Bob : c = 02f23656dce77507a6fb9d257594da05fb394966
        k = 13f6b41cb7c47c2ea63f8fe5fd14be142b6b8df7
        m = ceciestlemessageclairadechiffrer
```

Exécution 3

```
Alice : m = ceciestlemessageclairadechiffrer
        m_hex = 1104824a6b23092900c412c088806411d052c491
        k = 06ae29c5adf5b760e64d019df68e1b89e953ed68
        c = 17aaab8fc6d6be49e689135d7e0e7f98390129f9
Ève : c = 17aaab8fc6d6be49e689135d7e0e7f98390129f9
Bob : c = 17aaab8fc6d6be49e689135d7e0e7f98390129f9
        k = 06ae29c5adf5b760e64d019df68e1b89e953ed68
        m = ceciestlemessageclairadechiffrer
```

3. Message alternatif et preuve de secret parfait

```
{
  "m_prime": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
  "k_prime": "59f0ad43d567c4b15cbf9254783e0b1d83e0cb97",
  "verification": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"
}
```

On choisit $m' = a$ répété 32 fois. Son encodage est 0, donc $k' = c \text{ XOR } 0 = c$ et $D_2(k', c) = m'$. La même construction vaut pour n'importe quel message alternatif de 32 lettres : $k' = c \text{ XOR encode}(m')$.

Le secret parfait de Shannon signifie que, pour chaque cryptogramme de probabilité non nulle et tout message m , $\Pr[M=m \mid C=c] = \Pr[M=m]$. Avec une clé uniforme indépendante sur 160 bits, chaque m est associé à exactement une clé compatible avec c . Ainsi $\Pr[C=c \mid M=m] = 2^{(-160)}$, indépendamment de m . Le cryptogramme ne modifie donc pas les probabilités a priori des messages. L'exemple alternatif illustre cette ambiguïté ; l'argument d'uniformité et d'indépendance établit la propriété.

2.2 Réutilisation de la clé

1. Annulation de la clé

```
c1 XOR c2 = (m1 XOR k) XOR (m2 XOR k)
            = m1 XOR m2 XOR (k XOR k)
            = m1 XOR m2
```

Valeur hexadécimale sur 240 bits :

```
3aa2ca740dcb90986418aa0e9faebeafe6d5096908ec47739cddc0000000
```


2. Glissement du mot connu

Eve2 découpe c1 XOR c2 en 48 blocs de 5 bits. À chacune des 40 positions possibles du mot de neuf lettres « protocole », elle effectue le XOR entre le mot encodé et les blocs correspondants. Elle retient les positions dont chaque résultat appartient à 0..25. Il s'agit d'une plausibilité alphabétique, distincte de la plausibilité linguistique.

Position dans m1	Fragment correspondant de m2
1	fachtcdsk
11	gbxtwxgmn
13	wrwggfhup
19	gofgqxryj
21	eeqgrrdbg
23	reradiman
25	qcdzmjhkh
30	eypqygaxy
31	gqnfkmsxy
36	bnsppvole
37	tnsizcole
38	tnveocole
39	tkztocole

Treize positions passent ce premier filtre. La position 23 produit « reradiman », compatible avec la fin de « démarrera dimanche ». Le choix de cette position est une hypothèse linguistique, confirmée ensuite par la reconstruction cohérente et le rechiffrement des deux textes. Un filtre alphabétique seul ne suffit pas à la sélectionner.

3. Extension de la solution

À la position 18, on propose « demarreradimanche » dans m2. Le XOR impose « entleprotocoles » dans l'autre message.

À la position 23, on propose « protocoles securite » dans m1. Le XOR impose « reradimancheaminuit » dans l'autre message.

À la position 0, on propose « modifiezimmédiatement » dans m1. Le XOR impose « leserveurcentralredem » dans l'autre message.

L'extension du premier fragment donne « ...entleprotocoles... ». Le groupe « protocole de sécurité » impose « ...rera dimanche à minuit ». La proposition « modifiez immédiatement » au début du premier texte fait apparaître « le serveur central redém... » dans le second. Les extensions s'accordent et conduisent aux deux phrases ci-dessous.

```
m1 = modifiezimmédiatementleprotocoles securitexxxxxx
m2 = leserveurcentralredemarreradimancheaminuitxxxxxx
k  = f5289bcb08154b7fc6aaca0684ece215d8fde5f67e5184521147bbab9e5d
```

Lecture : « Modifiez immédiatement le protocole de sécurité » et « Le serveur central redémarrera dimanche à minuit ». Chaque phrase comporte 42 lettres et reçoit six x de bourrage à droite.

La clé se calcule comme $k = c1 \text{ XOR encode}(m1)$. Les deux assertions $E2(k,m1)=c1$ et $E2(k,m2)=c2$ passent. Ces vérifications démontrent la compatibilité exacte des messages proposés. À elles seules, elles ne prouvent pas une unicité mathématique parmi tous les couples possibles ; la langue et le mot connu permettent de reconnaître cette reconstruction.

4. Hypothèse violée

Le secret parfait du masque jetable suppose une clé uniforme indépendante, secrète et utilisée une seule fois. La réutilisation fait apparaître $m_1 \oplus m_2$; la clé n'est plus indépendante des observations conditionnées par le premier message. Le théorème portant sur une utilisation ne garantit pas le secret de plusieurs messages sous une même clé.

2.3 Interprétation

Sans mot connu, le XOR révèle toujours une relation entre les messages, sans donner automatiquement chaque texte. Les contraintes de langue, de format et de bourrage peuvent encore permettre une attaque par hypothèses. Un bloc nul indique des lettres identiques aux deux positions, et non nécessairement des x.

Si Ève connaît m_1 en entier, elle obtient $k = c_1 \text{ XOR encode}(m_1)$, puis $m_2 = c_2 \text{ XOR } k$. Elle peut déchiffrer tout autre cryptogramme utilisant la même clé sur les positions couvertes. Il faut une nouvelle clé indépendante et uniforme, aussi longue que le message, pour chaque chiffrement. Le XOR seul n'assure pas l'authenticité du message.

3. Un MAC défaillant

3.1 Code et exemples

Gen tire une clé uniforme de 64 bits. MAC conserve les 32 bits de poids faible du XOR. Verif compare le tag reçu au tag recalculé et retourne 1 ou 0. Le masquage par 0xffffffff équivaut ici au modulo 2^{32} .

```
{
  "k": "bb8f9f5a3fca67d8",
  "m": "0000000000000000",
  "t": "3fca67d8",
  "v_valide": 1,
  "v_invalide": 0
}
```

3.2 Traces

Clé commune : 9deab5093d00bd90. Chaque message ci-dessous apparaît en binaire sur exactement 64 bits et en hexadécimal sur 16 chiffres. Ève transmet la paire sans modification.

Message 1

```
m_bits = 0000000000000000000000000000000000000000000000000000000000000000
m_hex   = 0000000000000000
Alice   : m = 0000000000000000, k = 9deab5093d00bd90, t = 3d00bd90
Ève     : m = 0000000000000000, t = 3d00bd90
Bob      : m = 0000000000000000, k = 9deab5093d00bd90
          t = 3d00bd90, v = 1
```

Message 2

[illegible]

Message 3

[illegible]

Message 4

[illegible]

3.3 Modification en transit

Message initial	Message substitué	Tag conservé	Vérification
0000000000000000	8000000000000000	3d00bd90	1
8000000000000000	0000000000000000	3d00bd90	1
00000000ffffff	80000000ffffff	c2ff426f	1
aaaaaaaaaaaaaaaa	2aaaaaaaaaaaaaaaa	97aa173a	1

Dans ces exemples, Ève inverse uniquement le bit de poids fort : $m' = m \text{ XOR } 2^{63}$. Le tag dépend uniquement des 32 bits faibles, donc Bob accepte la modification sans que la clé soit connue d'Ève.

```
m' = m XOR (u << 32),  0 <= u < 2^32
MAC(k, m') = MAC(k, m)
```

Pour une clé et un tag fixés, les bits faibles du message sont imposés tandis que les 32 bits forts sont libres. Bob accepte exactement 2^{32} messages distincts, en comptant m.

3.4 Forgerie

On note $\text{low32}(x)$ les 32 bits faibles de x . De $t_{\text{obs}} = \text{low32}(m_{\text{obs}}) \text{ XOR } \text{low32}(k^*)$, Ève déduit $\text{low32}(k^*) = t_{\text{obs}} \text{ XOR } \text{low32}(m_{\text{obs}})$. Elle peut ensuite fabriquer le tag de tout message cible.

```
t_cible = low32(m_cible) XOR t_obs XOR low32(m_obs)
m_obs   = 34de51e80a5b0373
t_obs   = 4661f080
m_cible = f330a664f8b1ecd2
low32(k*) = 4c3af3f3
tag forqé = b48b1f21
```

Cette attaque permet une forgerie universelle : après une seule paire observée, Ève peut produire un tag correct pour n'importe quel nouveau message cible. Elle réalise donc aussi une forgerie existentielle et peut satisfaire une cible fixée d'avance. Sa probabilité de succès est 1 pour cette construction.

Clé compatible	Verif sur observation	Verif sur cible et tag forgé
000000004c3af3f3	1	1
000000014c3af3f3	1	1

Les deux clés diffèrent dans leur partie haute. Elles ont la même partie basse, révélée par l'observation, et les vérifications ont été exécutées dans le script.

3.5 Interprétation

La paire observée révèle exactement les 32 bits faibles de k^* , de valeur 4c3af3f3. Les 32 bits forts restent libres : 2^{32} clés sont compatibles. Ces bits inconnus n'offrent aucune sécurité, car le calcul du tag les ignore entièrement.

L'infalsifiabilité sous attaque à messages choisis est violée : une seule requête de tag suffit à calculer des tags valides pour de nouveaux messages. L'attaque fonctionne même avec une simple observation passive d'une paire. Pour la confidentialité d'un message, le XOR avec un masque uniforme ne révèle pas le message. Pour l'authentification, le message et le tag sont publics : un tag égal au XOR d'un message connu et de la clé révèle la partie utile de la clé. De plus, ce MAC ignore la moitié du message. La confidentialité et l'authenticité nécessitent des propriétés différentes.

Bonus : Vigenère

Longueur de clé : Kasiski et indice de coïncidence

kasiski recense les positions des trigrammes répétés et leurs distances successives. De nombreuses distances sont multiples de 7, par exemple DKQ à 96 et 103 (distance 7), BPI à 182 et 266 (84), et BOJ à 234 et 353 (119). Certaines répétitions peuvent être fortuites : EBN donne 333, non divisible par 7. Il ne faut donc pas exiger que toutes les distances aient le même diviseur.

Pour une colonne de N lettres, $IC = \text{somme } n_a(n_a-1) / [N(N-1)]$. EveVigenere calcule l'IC moyen des colonnes pour chaque longueur candidate de 1 à 12. La longueur attendue entre 5 et 7 est confirmée par un pic net à 7.

Longueur	IC moyen	Clé estimée par χ^2
1	0.04276	b
2	0.04282	pb
3	0.04146	bcb
4	0.04319	pbgb
5	0.04470	qxbha
6	0.04117	ubpbqb
7	0.08053	beejkqr
8	0.04282	pbnbpqb
9	0.04106	atjbpbqdk
10	0.04329	ucdbobjnhb
11	0.04228	exoachxjrhb
12	0.04260	poimqcubrnvb

Analyse des colonnes et résultat

Pour chaque colonne, les 26 décalages sont comparés aux fréquences françaises fournies par une statistique χ^2 . Les fréquences de référence sont normalisées, car les pourcentages arrondis ne totalisent pas exactement 100. Le décalage minimisant le score donne la lettre de clé de cette colonne. Cette estimation est heuristique ; le texte lisible et le rechiffrement exact servent de validation.

Clé : beejkqr

Texte sans accents ni espaces :

lereseauelectriquedelaregionasubiunepanneimportanteapreslatempetedeverglasdumoisdernier
nattaquantpatientpeutessayertouteslesclespossiblesmaislespacedesclesestparfoistropvastep
ourcelaleprogrammefichierligneparlignecomptelestlettresetafficheuntablaudesfrequenc
esalecranunesignaturenumeriquepermetdeprouverquunmessageprovientbiendesonauteuretqu'ilnap
asetemodifieencheminpourverifierlintegritedunmessageonajouteuncodedauthentificationcalcu
leapartirduneclesecrete

Le réseau électrique de la région a subi une panne importante après la tempête de verglas du mois dernier. Un attaquant patient peut essayer toutes les clés possibles mais l'espace des clés est parfois trop vaste pour cela. Le programme lit le fichier ligne par ligne, compte les lettres et affiche un tableau des fréquences à l'écran. Une signature numérique permet de prouver qu'un message provient bien de son auteur et qu'il n'a pas été modifié en chemin. Pour vérifier l'intégrité d'un message, on ajoute un code d'authentification calculé à partir d'une clé secrète.

Le script vérifie que le chiffrement du texte de 463 lettres avec la clé beejkqr restitue exactement le cryptogramme de la fiche.

Discussion, limites et sources

Ces implémentations servent à expliquer les schémas du devoir. Le César alterné et la substitution restent vulnérables à la structure de la langue. Le lexique d'Ève pour César est réduit : le nombre de candidats dépend du critère choisi. La cryptanalyse de substitution et la reconstruction des deux textes OTP utilisent des hypothèses linguistiques explicites, puis des contrôles de rechiffrement. Elles ne démontrent pas l'unicité de chaque solution dans tous les espaces de messages possibles.

La permutation de substitution demeure indéterminée sur trois lettres absentes : six clés complètes conviennent. Le masque jetable nécessite une distribution sûre de clés longues et une utilisation unique. Le MAC étudié ne convient pas à un usage réel. En pratique, il faut des primitives d'authentification étudiées et des schémas de chiffrement authentifié, et gérer soigneusement les clés et les valeurs à usage unique.

Les assertions exécutées contrôlent les empreintes, les allers-retours de chiffrement, le rechiffrement exact des textes de substitution et de Vigenère, les deux cryptogrammes OTP réutilisés, les tags MAC, les modifications acceptées et les deux clés compatibles. Le script contient aussi un cas explicite où Verif retourne 0. Les exemples aléatoires consignés proviennent d'une exécution réelle du script.

Sources consultées

1. Fiset, Martin. IFT814 - Cryptographie, Devoir 1, Automne 2026. Énoncé du devoir, version autorisant l'utilisation de l'IA, cinq pages. Source des questions, conventions et fréquences de référence.
2. Fiche de paramètres Equipe_03.txt, fournie avec le devoir. Source des clés, cryptogrammes et empreintes.
3. Fiset, Martin. Cours 1 : La Crypto. Support de cours, notamment les sections sur Kerckhoffs, les substitutions, le secret parfait et le masque jetable.
4. Fiset, Martin. Cours 2 : La Crypto et Cours 3 : Crypto Symétrique. Supports de cours, utilisés pour situer les attaques et l'authentification.

Annexe : code source Python complet

Le code source est reproduit ci-dessous en police à chasse fixe. Les données de la fiche Equipe_03.txt interviennent dans les traces présentées dans les différentes sections. Les lignes longues peuvent se poursuivre sur la ligne suivante pour les besoins de la mise en page.

```
"""IFT814, devoir 1. Implémentations pédagogiques, sans bibliothèque de chiffrement.
Paramètres : Equipe_03.txt, placé dans le même dossier.
Exécution : python solution_ifft814.py > resultats.json
Ce fichier produit des résultats vérifiables, pas un outil de sécurité réel.
"""

import collections
import hashlib
import json
import math
import re
import secrets
from pathlib import Path

ALPHABET = 'abcdefghijklmnopqrstuvwxyz'
MESSAGE = 'ceciestlemessagedclairdechiffrer'
FREQ = [7.6,.9,3.3,3.7,14.7,1.1,.9,.7,7.5,.6,.05,5.5,3,7.1,5.8,2.5,1.4,6.7,7.9,7.2,6.3,1.8,
.07,.4,.1,.3]
FREQ = [v/sum(FREQ) for v in FREQ]

def lettres(m):
    if not isinstance(m, str) or not m or any(a not in ALPHABET for a in m):
        raise ValueError('Message non vide en lettres a-z uniquement.')
    return m

def entier(x, n):
    if type(x) is not int or not 0 <= x < (1 << n):
        raise ValueError(f'Entier attendu sur {n} bits.')
    return x

def Gen1():
    return secrets.randbelow(26), secrets.randbelow(26)

def E1(k, m):
    lettres(m)
    if len(m) != 32 or len(k) != 2 or any(type(x) is not int or not 0 <= x < 26 for x in
k):
        raise ValueError('César : 32 lettres et deux décalages de 0 à 25.')
    return ''.join(ALPHABET[(ord(a)-97+k[i % 2]) % 26] for i,a in enumerate(m))

def D1(k, c):
    return E1(tuple((-a) % 26 for a in k), c)

# Petit lexique pédagogique fixé avant l'attaque. Ne contient pas le message entier.
LEXIQUE = set('a au aux ce ceci cela cette ces de des du le la les un une est sont et ou
que qui pour par sur dans avec sans message texte clair secret cle cles chiffrement
dechiffrer chiffrer bonjour monde il elle nous vous on il faut tester securite ordinateur
internet protocole alice bob eve'.split())

def segmenter(texte, lexique=LEXIQUE):
    # Programmation dynamique : on recherche une couverture totale par des mots.
    chemins = {0: []}
    for i in range(len(texte)):
        if i not in chemins:
            continue
        for j in range(i+1, len(texte)+1):
            if texte[i:j] in lexique and j not in chemins:
                chemins[j] = chemins[i] + [texte[i:j]]
    return chemins.get(len(texte))

def Evel(c):
    retenus = []
    for a in range(26):
        for b in range(26):
            candidat = D1((a,b), c)
            mots = segmenter(candidat)
            if mots:
                retenus.append({'cle':[a,b], 'texte':candidat, 'mots':mots})
    return retenus

def Gen3():
    # Fisher-Yates : chaque échange utilise une source uniforme sûre.
    p = list(ALPHABET)
    for i in range(25,0,-1):
        j = secrets.randbelow(i+1)
        p[i],p[j] = p[j],p[i]
    return ''.join(p)
```

```

def permutation(k):
    if not isinstance(k,str) or len(k) != 26 or set(k) != set(ALPHABET):
        raise ValueError('La clé doit être une permutation de a-z.')
    return k
def E3(k,m):
    permutation(k); lettres(m)
    return ''.join(k[ord(a)-97] for a in m)
def D3(k,c):
    permutation(k); lettres(c)
    inverse = ['']*26
    for i,a in enumerate(k):
        inverse[ord(a)-97] = ALPHABET[i]
    return ''.join(inverse[ord(a)-97] for a in c)
def frequences(c, n=1):
    lettres(c)
    if not 1 <= n <= len(c):
        raise ValueError('Taille de groupe invalide.')
    comptes = collections.Counter(c[i:i+n] for i in range(len(c)-n+1))
    if n == 1:
        comptes.update({a:0 for a in ALPHABET})
    return [{ 'groupe':a, 'nombre':v, 'pourcentage':100*v/(len(c)-n+1)} for a,v in
comptes.most_common()]
def partiel(c, correspondances):
    return ''.join(correspondances.get(a,'_') for a in c)
def vers_bits(m):
    lettres(m)
    valeur = 0
    for a in m:
        valeur = (valeur << 5) | (ord(a)-97)
    return valeur
def vers_lettres(v,n):
    entier(v,5*n)
    blocs = [(v >> (5*(n-1-i))) & 31 for i in range(n)]
    if any(a > 25 for a in blocs):
        raise ValueError('Bloc 5 bits hors alphabet (26 à 31).')
    return ''.join(ALPHABET[a] for a in blocs)
def Gen2(n=32):
    if type(n) is not int or n <= 0:
        raise ValueError('Longueur positive attendue.')
    return secrets.randbits(5*n)
def E2(k,m):
    lettres(m); entier(k,5*len(m))
    return vers_bits(m) ^ k
def D2(k,c,n=32):
    entier(k,5*n); entier(c,5*n)
    return vers_lettres(c ^ k,n)
def Eve2(c1,c2,mot='protocole',n=48):
    lettres(mot); entier(c1,5*n); entier(c2,5*n)
    delta=c1^c2
    blocs=[(delta >> (5*(n-1-i))) & 31 for i in range(n)]
    retenus=[]
    for pos in range(n-len(mot)+1):
        vals=[blocs[pos+i] ^ (ord(a)-97) for i,a in enumerate(mot)]
        if all(v < 26 for v in vals):
            retenus.append({'position':pos, 'fragment_m2': ''.join(ALPHABET[v] for v in
vals)})
    return retenus
def verifier_hypothese_otp(m1,m2,c1,c2):
    # Ne suffit pas à établir que des phrases candidates sont les phrases d'origine.
    if len(m1) != 48 or len(m2) != 48:
        raise ValueError('Deux textes de 48 lettres sont requis.')
    k=c1 ^ vers_bits(m1)
    return { 'cle':format(k,'060x'), 'c1_valide':E2(k,m1)==c1,
            'c2_valide':E2(k,m2)==c2, 'compatibles':(vers_bits(m1)^vers_bits(m2))==c1^c2}
def Gen():
    return secrets.randbits(64)
def MAC(k,m):
    entier(k,64); entier(m,64)
    return (m ^ k) & 0xffffffff
def Verif(k,m,t):
    entier(t,32)
    return int(MAC(k,m)==t)
def EveMAC(m_obs,t_obs,m_cible):
    entier(m_obs,64); entier(t_obs,32); entier(m_cible,64)

```

```

k_bas=(m_obs & 0xffffffff)^t_obs
return k_bas, (m_cible & 0xffffffff)^k_bas
def GenVigenere(n):
    return ''.join(ALPHABET[secrets.randbelow(26)] for _ in range(n))
def Vigenere(k,m,dechiffrement=False):
    lettres(k); lettres(m)
    signe=-1 if dechiffrement else 1
    return ''.join(ALPHABET[(ord(a)-97+signe*(ord(k[i%len(k)))-97))%26] for i,a in
enumerate(m))
def indice(c):
    n=len(c)
    return sum(v*(v-1) for v in collections.Counter(c).values())/(n*(n-1)) if n>1 else 0
def kasiski(c):
    groupes=collections.defaultdict(list)
    for i in range(len(c)-2):
        groupes[c[i:i+3]].append(i)
    return [{'trigramme':a,'positions':p,'distances':[p[i+1]-p[i] for i in
range(len(p)-1)]]
        for a,p in sorted(groupe.items()) if len(p)>1]
def EveVigenere(c):
    essais=[]
    for n in range(1,13):
        colonnes=[c[i::n] for i in range(n)]
        cle=''
        for col in colonnes:
            scores=[]
            for k in range(26):
                counts=collections.Counter((ord(a)-97-k)%26 for a in col)
                scores.append(sum((counts.get(a,0)-len(col)*FREQ[a])**2/(len(col)*FREQ[a])
for a in range(26)))
            cle+=ALPHABET[min(range(26), key=lambda k:scores[k])]
        essais.append({'longueur':n,'ic_moyen':sum(indice(col) for col in colonnes)/n,
            'cle':cle,'texte':Vigenere(cle,c,True)})
    return essais
def charger(path):
    s=Path(path).read_text()
    get=lambda pattern: re.search(pattern,s).group(1)
    return {'cesar':[(int(a),int(b)) for a,b in re.findall(r'k1 = (\d+), k2 = (\d+)',s)],
        'sub':re.findall(r'Exécution \d : ([a-z]{26})',s),
        'otp':int(v,16) for v in re.findall(r'Exécution \d : ([0-9a-f]{40})',s)],
        'sub_c':get(r'\n ([A-Z]{398})').lower(),
        'vigenere':get(r'\n ([A-Z]{463})').lower(),
        'c1':int(get(r'c1 = (\w+)',16)), 'c2':int(get(r'c2 = (\w+)',16)),
        'mac_k':int(get(r' k = (\w+)',16)),
        'm_obs':int(get(r'm_obs = (\w+)',16)), 't_obs':int(get(r't_obs = (\w+)',16)),
        'm_cible':int(get(r'm_cible = (\w+)',16))
def main(path):
    p=charger(path);r={}
    r['empreintes']={'sub':hashlib.sha256(p['sub_c'].encode()).hexdigest()[8:],
        'otp':hashlib.sha256((format(p['c1'],'060x')+format(p['c2'],'060x')).encode()).hexdigest
est()[8:],
        'vigenere':hashlib.sha256(p['vigenere'].encode()).hexdigest()[8:]}
    assert r['empreintes']=={'sub':'2b86f4d7','otp':'d7b627e1','vigenere':'768d2dfa'}
    r['cesar']=[]
    for k in p['cesar']:
        c=E1(k,MESSAGE);m=D1(k,c);assert m==MESSAGE
        r['cesar'].append({'m':MESSAGE,'k':k,'c':c,'bob':m,'candidats':Evel(c)})
    r['sub_traces']=[]
    for k in p['sub']:
        c=E3(k,MESSAGE);m=D3(k,c);assert m==MESSAGE
        r['sub_traces'].append({'m':MESSAGE,'k':k,'c':c,'bob':m})
    mp=dict(zip('YAOEJGBWHKFIXPDRTZSCUQV'.lower(),['depuiscclgnrxtomhqvfyb']))
    clair=partiel(p['sub_c'],mp);assert '_' not in clair
    # Les lettres k,w,z sont absentes : completion choisie, sans prétendre à l'unicité.
    complet=mp|{'l':'k','m':'w','n':'z'}
    k=''.join(next(a for a,v in complet.items() if v==b) for b in ALPHABET)
    assert E3(k,clair)==p['sub_c']
    stages=[{'a':'e','g':'s','w':'l'}, {a:mp[a] for a in 'yaoejgbw'},mp]
    r['sub_analyse']={'frequences':frequences(p['sub_c']),
'bigrammes':frequences(p['sub_c'],2)[15],
        'trigrammes':frequences(p['sub_c'],3)[10],
'etapes': [{'correspondances':m,'texte':partiel(p['sub_c'],m)} for m in stages],
        'clair':clair,'cle_compatible':k,'lettres_chiffrees_absentes':'lmn','lettres_claires_

```



```

absentes': 'kwz',
    'nombre_completions': 6, 'reencodage_exact': True, 'espace_cles': math.factorial(26), 'bits_
_cles': math.log2(math.factorial(26))}
    mb=vers_bits(MESSAGE); r['otp_traces']=[]
    for k in p['otp']:
        c=E2(k,MESSAGE); m=D2(k,c); assert m==MESSAGE
    r['otp_traces'].append({'m':MESSAGE, 'm_hex':format(mb, '040x'), 'm_bits':format(mb, '0160b'),
        'k':format(k, '040x'), 'c':format(c, '040x'), 'bob':m})
    autre='a'*32; c=E2(p['otp'][0],MESSAGE); kp=c^vers_bits(autre)
    assert D2(kp,c)==autre
    r['otp_alternatif']={'m_prime':autre, 'k_prime':format(kp, '040x'), 'verification':D2(kp,c)}
    r['otp_reutilisation']={'xor':format(p['c1']^p['c2'], '060x'),
    'candidats':Eve2(p['c1'],p['c2']),
        'statut':'reconstruction linguistique compatible, rechiffrement
vérifié'}
    m1='modifiezimmédiatementleprotocolede securite'.ljust(48, 'x')
    m2='leserveurcentralredemarreradimancheaminuit'.ljust(48, 'x')
    otp_verif=verifier_hypothese_otp(m1,m2,p['c1'],p['c2'])
    assert otp_verif['c1_valide'] and otp_verif['c2_valide']
    r['otp_reutilisation'].update({'m1':m1, 'm2':m2, 'verification':otp_verif})
    delta=p['c1']^p['c2']
    blocs=[(delta >> (5*(47-i))) & 31 for i in range(48)]
    expansions=[]
    for cote,pos,texte in [(m2,18,'demarreradimanche'),(m1,23,'protocolede securite'),
        (m1,0,'modifiezimmédiatement')]:
        fragment=''.join(ALPHABET[blocs[pos+i]^(ord(a)-97)] for i,a in enumerate(texte))
        expansions.append({'hypothese_sur':cote, 'position':pos, 'hypothese':texte, 'fragment_
autre':fragment})
    r['otp_reutilisation']['extensions']=expansions
    r['mac_traces']=[]
    for m in [0,1<<63,(1<<32)-1,0xffffffffffffffff]:
        t=MAC(p['mac_k'],m); mf=m^(1<<63)
        assert Verif(p['mac_k'],m,t)==1 and Verif(p['mac_k'],mf,t)==1
        assert Verif(p['mac_k'],m,t^1)==0
        r['mac_traces'].append({'m_hex':format(m, '016x'), 'm_bits':format(m, '064b'),
            'k':format(p['mac_k'], '016x'), 't':format(t, '08x'), 'v':1, 'm_prime':format(mf, '016x
'), 'v_prime':1})
    kb,t=EveMAC(p['m_obs'],p['t_obs'],p['m_cible']); cles=[kb,(1<<32)|kb]
    for k in cles:
        assert Verif(k,p['m_obs'],p['t_obs'])==1 and Verif(k,p['m_cible'],t)==1
    r['mac_forgerie']={'k_bas':format(kb, '08x'), 'tag':format(t, '08x'), 'cles_compatibles':[f
ormat(k, '016x') for k in cles]}
    r['vigenere']={'kasiski':kasiski(p['vigenere']), 'essais':EveVigenere(p['vigenere'])}
    meilleur=max(r['vigenere']['essais'],key=lambda a:a['ic_moyen'])
    assert Vigenere(meilleur['cle'],meilleur['texte'])==p['vigenere']
    r['vigenere']['retenu']=meilleur
    # Exemples effectifs des generateurs et de l'aller-retour associe.
    k1=Gen1(); k3=Gen3(); k2=Gen2(); km=Gen()
    r['exemples_aleatoires']={'Gen1':{'k':k1, 'c':E1(k1,MESSAGE), 'm_retrouve':D1(k1,E1(k1,ME
SSAGE))},
        'Gen3':{'k':k3, 'c':E3(k3,MESSAGE), 'm_retrouve':D3(k3,E3(k3,MESSAGE))},
        'Gen2':{'k':format(k2, '040x'), 'c':format(E2(k2,MESSAGE), '040x'), 'm_retrouve':D2(k2,E2(
k2,MESSAGE))},
        'Gen_MAC':{'k':format(km, '016x'), 'm':'0000000000000000', 't':format(MAC(km,0), '08x'),
            'v_valide':Verif(km,0,MAC(km,0)), 'v_invalide':Verif(km,0,MAC(km,0)^1)}}
    print(json.dumps(r,ensure_ascii=False,indent=2))
if __name__ == '__main__':
    import sys
    main(sys.argv[1] if len(sys.argv)>1 else Path(__file__).with_name('Equipe_03.txt'))

```